INTERNSHIP PROJECT REPORT

(Payload Generation Framework Development)

Organization

ITSOLERA Pvt. Ltd

Department

Offensive Security

Group/Team

Team Alpha

Date: February 16th, 2026

TABLE OF CONTENTS

ABSTRACT **2**
1. Introduction **3**
2. Project Objectives **3**
3. Project Scope **3**
4. System Requirements **4**
4.1. Hardware Requirements 4
4.2. Software Requirements 4
5. Tools & Technologies **4**
6. Architecture & Design **5**
6.1. High-Level Architecture 5
6.2. Modular Design Approach 5
6.3. Data Flow 5
7. Execution Workflow **6**
8. Implementation Details **6**
8.1. Directory & File Structure 6
8.2. Command-Line Interface 7
8.3. Output File Generation 7
9. Functional Workflow & Tool Execution **7**
9.1. Listing Modules and Showing Info 7
9.2. Generating XSS Payloads with Context Filtering 8
9.3. SQLi with Database and Type Filter 9
9.4. Command Injection - Windows, Random 2 Payloads 9
9.5. Encoding + Obfuscation + CSV Export 9
9.6. SSTI - Jinja2 Payloads to JSON 10
10. Tool Usage Scenarios **10**
11. Ethical Considerations **10**
12. Results & Outcomes **10**
13. Limitations & De-merits **11**
14. Future Enhancements **11**
15. Conclusion **11**
16. References **12**

ABSTRACT

In modern web application security testing, understanding how attackers craft malicious payloads and evade defensive controls is essential for building robust protections. Manual payload generation is time-consuming and often lacks systematic coverage of injection contexts, encoding techniques, and bypass logic. This project presents **VectorCraft**, an educational, modular payload generation framework that produces safe, non-executing payload templates for common web vulnerabilities: Cross-Site Scripting (XSS), SQL Injection (SQLi), Command Injection (CMDi), and Server-Side Template Injection (SSTI). The tool supports filtering by payload type, context, database, and platform, and applies encoding (URL, Base64, Hex, ROT13) and obfuscation (comment insertion, case variation, whitespace manipulation) to demonstrate evasion techniques. Output can be printed to the terminal or exported to JSON, TXT, or CSV. VectorCraft is strictly intended for defensive research, academic learning, and authorised testing environments, aligning with OWASP ethical standards.

1. Introduction

Security professionals and penetration testers frequently need to generate injection strings to validate input validation mechanisms, test Web Application Firewalls (WAFs), or demonstrate risks during training. While public cheat sheets (e.g., PayloadsAllTheThings) exist, they are static and lack programmatic filtering, transformation, and export capabilities. Moreover, manual adaptation of payloads for different contexts (HTML, attribute, JavaScript) or databases (MySQL, PostgreSQL, MSSQL, Oracle, SQLite) is error-prone and inefficient.

VectorCraft addresses these gaps by providing a command-line interface (CLI) that dynamically generates payload templates based on user-supplied filters, applies encoding/obfuscation to illustrate bypass logic, and exports results in structured formats. The tool is built as a modular Python framework, making it extensible and easy to maintain.

2. Project Objectives

The primary objectives of the VectorCraft project are:

- Design a **modular payload generation framework** supporting XSS, SQL Injection, Command Injection, and SSTI.
- Implement **context-aware filtering** (HTML/JS/attribute for XSS; database types for SQL Injection; platforms for Command Injection).
- Provide **encoding demonstrations** (URL, Base64, Hex, ROT13) to show how attackers obfuscate payloads.
- Provide **obfuscation techniques** (comment insertion, case randomisation, whitespace manipulation) that mimic real-world evasion.
- Offer **multiple export formats** (JSON, TXT, CSV) for integration with other tools (e.g., Burp Suite, OWASP ZAP).
- Maintain a **clear ethical disclaimer** - payloads are non-executing templates, and no live attack traffic is generated.

3. Project Scope

VectorCraft focuses exclusively on the **generation of payload templates** for educational and defensive research. The scope includes:

- **XSS Module**: Reflected, stored, and DOM-based payloads; contexts: HTML, attribute, JavaScript.
- **SQL Injection Module**: Error-based, union-based, boolean-based blind, time-based blind; databases: MySQL/MariaDB, PostgreSQL, MSSQL, Oracle, SQLite.
- **Command Injection Module**: Command separators (`&&`, `|`, `;`, ``, `$()`, newline, background) for Linux and Windows.

- **SSTI Module**: Templates for Jinja2, Twig, and Freemarker engines.
- **Transformations**: Encoding (URL, Base64, Hex, ROT13) and obfuscation (comment, case, whitespace).
- **Export**: CLI output, JSON, TXT, CSV.

The following are **explicitly out of scope**:

- Live execution of payloads against any target.
- Automated vulnerability scanning or exploitation.
- Network-level attacks or post-exploitation activities.

All generated payloads are clearly labelled as templates and include descriptive metadata.

4. System Requirements

4.1. Hardware Requirements

- **RAM**: 2 GB minimum (4 GB recommended)
- **Processor**: Dual-core CPU or higher
- **Storage**: 500 MB free disk space (for source code and exports)

4.2. Software Requirements

- **Operating System**: Linux (Kali Linux, Ubuntu), macOS, or Windows (with WSL)
- **Python**: Version 3.8 or higher
- **Dependencies**: `pyfiglet` (ASCII banner), `rich` (coloured console output)
- **Optional**: `git` for cloning the repository

5. Tools & Technologies

Technology	Purpose
Python 3.8+	Core programming language
`pyfiglet`	Rendering the startup banner
`rich`	Coloured logging, tables, and progress indicators
`argparse`	Command-line argument parsing
`json` / `csv`	Structured export formats
`urllib.parse`, `base64`, `codecs`	Encoding implementations
`random`, `re`	Obfuscation techniques

No third-party security tools (e.g., sqlmap, Burp Suite) are invoked - VectorCraft is self-contained.

6. Architecture & Design

VectorCraft follows a modular, layered architecture that separates payload definitions, transformation logic, and user interface.

6.1. High-Level Architecture

```
vectorcraft.py (entry point, banner)
        │
        ▼
source/cli.py (argument parsing, orchestration)
        │
        ├── modules/xss.py, sqli.py, cmdi.py, ssti.py (payload databases)
        ├── source/encoder.py (encoding transformations)
        ├── source/obfuscator.py (obfuscation transformations)
        ├── source/exporter.py (JSON/TXT/CSV export)
        ├── source/utils.py (logging, file writing)
        └── source/base.py (active configuration display)
```

6.2. Modular Design Approach

Each vulnerability class is implemented as an independent module under the modules/ directory. All modules expose a consistent `generate_payloads(filters=None)` function that returns a list of dictionaries, each containing at least type, payload, and description. This uniformity allows the orchestrator (cli.py) to dispatch calls without knowing module internals. Benefits of this design:

- Adding a new vulnerability (e.g., XXE, SSRF) requires only a new module file.
- Filters are module-specific and validated at the CLI level.
- Transformations (encoding/obfuscation) are applied after payload generation, keeping modules clean.

6.3. Data Flow

- User supplies CLI arguments (e.g., `-m xss --context html --encode url`).
- The `cli.py` parses and validates arguments, then prints the active configuration.
- The requested module's `generate_payloads()` is called with filters.
- The raw payload list is optionally randomised (`--random`), limited (`--count`), transformed (obfuscation, then encoding), and finally exported or printed.

7. Execution Workflow

When a user runs a command, the following steps occur sequentially:

- **Banner display** (unless `--no-banner` is used).
- **Argument parsing** and validation (e.g., `--db` only allowed with `sqli`).
- **Active configuration** is printed to the console.
- **Payload generation** - the module returns all matching payloads.
- **Random selection** (if `--random` given) - picks `N` distinct payloads.
- **Limit** (if `--count` given) - truncates the list.
- **Obfuscation** (if `--obfuscate` given) modifies payloads (case, space, comments).
- **Encoding** (if `--encode` given) - transforms the final payload strings.
- **Export or print** - writes to file or displays JSON on the terminal.

This workflow ensures that transformations are applied in a logical order (obfuscation first, then encoding) and that metadata (type, description, etc.) remains intact.

8. Implementation Details

8.1. Directory & File Structure

```
VectorCraft/
├── vectorcraft.py          # Main entry point (banner, CLI dispatch)
├── requirements.txt        # Python dependencies
├── README.md               # User documentation
├── source/                 # Core framework
│   ├── __init__.py
│   ├── cli.py              # Argument parsing and orchestration
│   ├── base.py             # Active configuration printer
│   ├── encoder.py          # URL/Base64/Hex/ROT13 encoding
│   ├── exporter.py         # JSON/TXT/CSV writers
│   ├── obfuscator.py       # Comment, case, whitespace obfuscation
│   └── utils.py            # Logging, coloured console, safe file write
└── modules/                # Payload databases
    ├── __init__.py
    ├── xss.py              # 20+ XSS payloads with context metadata
    ├── sqli.py             # 30+ SQLi payloads for 6 database types
    ├── cmdi.py             # 20+ command injection patterns
```

```
    └── ssti.py            # 9 SSTI payloads (Jinja2, Twig, Freemarker)
```

8.2. Command-Line Interface

The tool uses Python's `argparse` with grouped arguments for clarity. All core features are exposed via flags:

Group	Flag	Description
General	`-h, --help`	Show help
	`--list-modules`	List available modules and payload counts
	`--info MODULE`	Show detailed info for a module
Core	`-m, --module`	Select module (xss, sqli, cmdi, ssti)
	`--db`	Filter SQLi by database
	`--random [N]`	Pick N random payloads
	`--count N`	Limit output count
Filters	`--type`	Substring match on payload type
	`--context`	XSS injection context
	`--platform`	Linux/Windows/web
Transformations	`--encode`	url, base64, hex, rot13
	`--obfuscate`	comment, case, whitespace
Export	`--export`	json, txt, csv
	`--output`	Custom filename

8.3. Output File Generation

When `--export` is specified, the tool automatically creates a timestamped filename (e.g., `xss_payloads_2026-04-11_12-30-45.json`) unless `--output` overrides it. JSON exports preserve all metadata; TXT exports present a human-readable list; CSV exports flatten the payload dictionaries into columns. All writes use `safe_write_file()` which creates parent directories if they do not exist.

9. Functional Workflow & Tool Execution

The following examples demonstrate VectorCraft's capabilities. (Screenshots are available in the GitHub repository and demonstration video.)

9.1. Listing Modules and Showing Info

```
python vectorcraft.py --list-modules
```

Output (coloured): Available Modules:

- xss - 20 payloads
- sqli - 35 payloads
- cmdi - 20 payloads
- ssti - 9 payloads

```
python vectorcraft.py --info sqli
```

Output:

- Module: SQLI
- Filters:

`--type`	`--platform`	`--db` (required)

- Databases:

MySQL	postgres	MSSQL	MariaDB	SQLite3	Oracle

- Example usage:

```
python3 vectorcraft.py -m sqli --type 'error' --export json
```

9.2. Generating XSS Payloads with Context Filtering

```
python vectorcraft.py -m xss --context attribute
```

- **Output** (truncated):

```
[
  {
    "type": "Attribute Context Breakout",
    "payload": "\" onmouseover=\"alert(3)\"",
    "description": "Breaks out of HTML attribute context.",
    "platform": "web",
    "context": "attribute"
  },
  ...
]
```

9.3. SQLi with Database and Type Filter

```
python vectorcraft.py -m sqli --db oracle --type error
```

- **Output**:

```
[
  {
    "type": "Error-Based Injection",
    "payload": "' OR 1=1 -- ",
    "description": "Modifies query logic to evaluate true.",
    "database": "oracle",
    "platform": "linux"
  }
]
```

9.4. Command Injection - Windows, Random 2 Payloads

```
python vectorcraft.py -m cmdi --platform windows --random 2
```

- **Output** (example random selection):

```
[
  {
    "type": "Command Injection -- OR Operator",
    "payload": "|| whoami",
    "description": "Executes command when the first command fails.",
    "platform": "windows"
  },
  {
    "type": "Command Injection -- Pipe Operator",
    "payload": "| whoami",
    "description": "Passes output of one command into another.",
    "platform": "windows"
  }
]
```

9.5. Encoding + Obfuscation + CSV Export

```
python vectorcraft.py -m xss --encode base64 --obfuscate case --export csv
--output xss_encoded.csv
```

The resulting CSV file contains columns: type, payload (obfuscated then base64-encoded), description, platform, context.

9.6. SSTI - Jinja2 Payloads to JSON

- Creates a file `ssti_payloads_2026-04-11_15-22-10.json` with all Jinja2-related SSTI templates.

```
python vectorcraft.py -m ssti --type Jinja2 --export json
```

10. Tool Usage Scenarios

VectorCraft is suitable for the following authorised activities:

- **Security Training & CTFs**: Quickly provide trainees with contextual payload examples.
- **WAF Rule Testing**: Generate encoded/obfuscated payloads to validate detection logic.
- **Penetration Testing**: Use as a reference during manual testing (no live automation).
- **Defensive Research**: Understand how attackers mutate payloads to bypass filters.
- **Integration with BurpSuite/ZAP**: Export payloads in CSV/JSON and import into intruder or fuzzer.

11. Ethical Considerations

VectorCraft was developed strictly for **educational and defensive purposes**. Key ethical safeguards include:

- **No live attack traffic**: The tool never sends HTTP requests or executes commands.
- **Clear labelling**: All output is described as “payload templates” with disclaimers.
- **No automated scanning**: The user must manually copy payloads into their own authorised test environment.
- **OWASP alignment**: The project follows the OWASP Code of Ethics, emphasising responsible use.

Misuse of this tool (e.g., injecting payloads against systems without explicit permission) is strictly prohibited and the authors assume no liability.

12. Results & Outcomes

VectorCraft successfully delivers a modular, extensible payload generation framework that meets all specified requirements:

- **Four modules** (XSS, SQLi, CMDi, SSTI) with 84+ unique payload templates.
- **Filtering** by type, context, database, platform - all functional.
- **Random selection** (`--random`) and count limiting (`--count`) work as designed.

- **Encoding** (URL, Base64, Hex, ROT13) and **obfuscation** (comment, case, whitespace) applied correctly.
- **Export** to JSON, TXT, CSV with timestamped or custom filenames.
- **The help system** (`-h`, `--list-modules`, `--info`) provides clear guidance.

The tool has been tested on Kali Linux, Ubuntu, and Windows WSL environments. Performance is instantaneous (no network calls), making it suitable for rapid payload prototyping.

13. *Limitations & De-merits*

- **Static payload database** - Payloads are hard-coded; dynamic generation based on context analysis is not implemented.
- **Limited obfuscation depth** - Comment insertion is rule-based and may not bypass advanced WAFs.
- **No live validation** - The tool cannot confirm whether a payload would actually execute in a given environment.
- **Dependency on external libraries** - `pyfiglet` and `rich` must be installed; however, these are lightweight and widely available.
- **Platform-specific CMDi payloads** - Some Windows examples assume `cmd.exe` behaviour; PowerShell variants are not covered.

14. *Future Enhancements*

- **Dynamic payload generation** - Use templates with placeholders that are substituted based on user input (e.g., custom JavaScript functions).
- **WAF bypass simulation** - Integrate known bypass patterns (e.g., HTTP parameter pollution, Unicode normalisation) as configurable layers.
- **Live export to Burp Suite Intruder** - Generate payload lists in Burp's native format or via REST API.
- **Additional modules**: XXE, SSRF, LDAP injection, NoSQL injection.
- **Graphical user interface (GUI)**: A web-based dashboard for payload preview and export.
- **YAML configuration**: Allow users to add custom payloads without editing Python code.

15. *Conclusion*

VectorCraft demonstrates how an educational payload generation framework can bridge the gap between theoretical vulnerability knowledge and practical testing. By offering

modular, filterable, and transformable payload templates, the tool empowers security learners and practitioners to understand injection contexts, encoding strategies, and obfuscation techniques - all without executing any harmful code. The project successfully fulfils the internship objectives and provides a solid foundation for future extensions. VectorCraft stands as a responsible, open-source contribution to the defensive security community.

16. References

- OWASP Foundation. (n.d.). *Cross-site scripting (XSS)*. Retrieved from https://owasp.org/www-community/attacks/xss/
- OWASP Foundation. (n.d.). *SQL injection*. Retrieved from https://owasp.org/www-community/attacks/SQL_Injection
- OWASP Foundation. (n.d.). *Command injection*. Retrieved from https://owasp.org/www-community/attacks/Command_Injection
- OWASP Foundation. (n.d.). *Server-side template injection*. Retrieved from https://owasp.org/www-community/attacks/Server_Side_Template_Injection
- PortSwigger. (n.d.). *Cross-site scripting cheat sheet*. Retrieved from https://portswigger.net/web-security/cross-site-scripting/cheat-sheet
- PayloadsAllTheThings Contributors. (n.d.). *PayloadsAllTheThings*. Retrieved from https://github.com/swisskyrepo/PayloadsAllTheThings
- Python Software Foundation. (2026). *argparse — Parser for command-line options*. Retrieved from https://docs.python.org/3/library/argparse.html

THE END